Informe Triggers Módulo 2

Gestión de Activos Biológicos

Versión: 1.0

27 de abril de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TRG-M2-01 — Coherencia de tipo de activo (INDIVIDUAL vs POBLACIONAL)

Descripción	Válida coherencia entre tipo y los campos indentficador / cantidad_inicial. Para INDIVIDUAL: identificador obligatorio, la cantidad debe ser nula. Para POBLACIONAL: cantidad obligatoria y > 0, identificador debe ser nulo. Rechaza combinaciones inválidas.
Aplicado a	RF-33
Restricción origen RF	"Si tipo_activo = INDIVIDUAL: identificador es obligatorio, cantidad_inicial debe ser null. Si tipo_activo = POBLACIONAL: cantidad_inicial es obligatoria, identificador debe ser null. El sistema debe rechazar cualquier combinación inválida. No se permite el envío simultáneo de identificador y cantidad_inicial." *(RF-33 — Restricciones)*
Justificación	Es un invariante estructural de la entidad. Si el backend falla, un INSERT directo con combinación inválida (ambos campos presentes o ambos nulos) persistiría un activo en estado incoherente que ningún módulo posterior podría procesar correctamente.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_activo_biologico_coherencia_tipo()
RETURNS TRIGGER AS $$
BEGIN
    -- Impedir que ambos campos estén presentes simultáneamente
    IF NEW.indentficador IS NOT NULL AND
       (NEW.atributos_dinamicos->>'cantidad_inicial') IS NOT NULL
THEN
        RAISE EXCEPTION 'INVALID_COMBINATION: No se permite
registrar identificador y cantidad_inicial simultáneamente. Tipo
```

```
de activo: %.',
            NEW.tipo
            USING ERRCODE = 'P0201';
    END IF;

    IF NEW.tipo = 'INDIVIDUAL' THEN
        IF NEW.indentficador IS NULL OR TRIM(NEW.indentficador) =
'' THEN
            RAISE EXCEPTION 'MISSING_FIELD: Para activos de tipo
INDIVIDUAL el identificador es obligatorio.'
                USING ERRCODE = 'P0202';
        END IF;
        -- cantidad_inicial debe ser nula para INDIVIDUAL
        IF (NEW.atributos_dinamicos->>'cantidad_inicial') IS NOT
NULL THEN
            RAISE EXCEPTION 'INVALID_COMBINATION: Para activos
INDIVIDUAL el campo cantidad_inicial debe ser nulo.'
                USING ERRCODE = 'P0202';
        END IF;

    ELSIF NEW.tipo = 'POBLACIONAL' THEN
        IF (NEW.atributos_dinamicos->>'cantidad_inicial') IS NULL
THEN
            RAISE EXCEPTION 'MISSING_FIELD: Para activos de tipo
POBLACIONAL la cantidad_inicial es obligatoria.'
                USING ERRCODE = 'P0203';
        END IF;
        IF (NEW.atributos_dinamicos->>'cantidad_inicial')::INTEGER
<= 0 THEN
            RAISE EXCEPTION 'INVALID_VALUE: La cantidad_inicial
debe ser mayor a cero. Valor recibido: %.',
                NEW.atributos_dinamicos->>'cantidad_inicial'
                USING ERRCODE = 'P0203';
        END IF;
        IF NEW.indentficador IS NOT NULL THEN
            RAISE EXCEPTION 'INVALID_COMBINATION: Para activos
POBLACIONAL el identificador debe ser nulo.'
                USING ERRCODE = 'P0203';
        END IF;
    END IF;

    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_biologico_coherencia_tipo
BEFORE INSERT ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION
modulo2.trg_fn_activo_biologico_coherencia_tipo();
```

TRG-M2-02 — Estado inicial forzado a ACTIVO

Descripción	Fuerza que id_estado apunte siempre al estado ACTIVO al momento del registro, independientemente del valor enviado por el usuario o el backend. El estado inicial no es configurable externamente.
Aplicado a	RF-33
Restricción origen RF	"El estado del activo será asignado automáticamente por el sistema con valor ACTIVO al momento del registro y no es configurable por el usuario." *(RF-33 — Restricciones)*
Justificación	Es la garantía más básica del ciclo de vida del activo. Sin este trigger, un INSERT directo podría crear activos en estado BAJA o CERRADO, corrompiendo el modelo de estados desde el origen.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_activo_biologico_estado_inicial()
RETURNS TRIGGER AS $$
DECLARE
    v_id_estado_activo INTEGER;
BEGIN
    SELECT id_estado_activo_biologico INTO v_id_estado_activo
    FROM modulo2.estados_activos_biologicos
    WHERE UPPER(nombre) = 'ACTIVO'
    LIMIT 1;
```

```
    IF v_id_estado_activo IS NULL THEN
        RAISE EXCEPTION 'CONFIG_ERROR: El estado ACTIVO no existe en el
catálogo de estados del activo biológico.'
            USING ERRCODE = 'P0204';
    END IF;

    NEW.id_estado := v_id_estado_activo;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_biologico_estado_inicial
BEFORE INSERT ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_activo_biologico_estado_inicial();
```

TRG-M2-03 — Validación de origen financiero y costo de adquisición

Descripción	Valida las reglas del campo origen_financiero: COMPRA/DONACION exigen costo_adquisicion > 0; NACIMIENTO exige ambos campos nulos; TRANSFERENCIA_INTERNA permite costo opcional pero si se registra debe ser > 0.
Aplicado a	RF-33
Restricción origen RF	"Si origen_financiero_activo = COMPRA o DONACION: costo_adquisicion es obligatorio y debe ser > 0. soporte_documental_costo es obligatorio. Si origen_financiero_activo = NACIMIENTO: costo_adquisicion debe ser nulo. soporte_documental_costo debe ser nulo. Si origen_financiero_activo = TRANSFERENCIA_INTERNA: costo_adquisicion es opcional. Si se registra, debe ser > 0 y llevar soporte_documental_costo." *(RF-33 —*

	Restricciones)
Justificación	El costo_adquisicion es inmutable (RF-33 lo define así) y es base para RF-82. Si un activo se persiste con un costo incoherente con su origen financiero, los módulos contables quedan comprometidos irreversiblemente.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_activo_biologico_origen_financiero()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.origen_financiero IN ('COMPRA', 'DONACION') THEN
        IF NEW.costo_adquisicion IS NULL OR NEW.costo_adquisicion <= 0
THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: El origen financiero %
exige un costo_adquisicion mayor a cero. Valor recibido: %.',
                NEW.origen_financiero, NEW.costo_adquisicion
                USING ERRCODE = 'P0205';
        END IF;
        -- Nota: soporte_documental está en atributos_dinamicos o en
columna separada.
        -- Se valida contra atributos_dinamicos hasta que el DDL agregue
la columna.
        IF (NEW.atributos_dinamicos->>'soporte_documental') IS NULL
           OR TRIM(NEW.atributos_dinamicos->>'soporte_documental') = ''
THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: El origen financiero %
exige soporte documental registrado.',
                NEW.origen_financiero
                USING ERRCODE = 'P0205';
        END IF;

    ELSIF NEW.origen_financiero = 'NACIMIENTO' THEN
        IF NEW.costo_adquisicion IS NOT NULL THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: Para origen NACIMIENTO el
costo_adquisicion debe ser nulo. Valor recibido: %.',
                NEW.costo_adquisicion
                USING ERRCODE = 'P0206';
        END IF;
        IF (NEW.atributos_dinamicos->>'soporte_documental') IS NOT NULL
THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: Para origen NACIMIENTO el
soporte_documental debe ser nulo.'
                USING ERRCODE = 'P0206';
```

```
        END IF;

    ELSIF NEW.origen_financiero = 'TRANSFERENCIA_INTERNA' THEN
        IF NEW.costo_adquisicion IS NOT NULL AND NEW.costo_adquisicion
<= 0 THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: Si se registra
costo_adquisicion en TRANSFERENCIA_INTERNA, debe ser mayor a cero. Valor
recibido: %.',
                NEW.costo_adquisicion
                USING ERRCODE = 'P0207';
        END IF;
        IF NEW.costo_adquisicion IS NOT NULL
           AND ((NEW.atributos_dinamicos->>'soporte_documental') IS NULL
                OR TRIM(NEW.atributos_dinamicos->>'soporte_documental')
= '') THEN
            RAISE EXCEPTION 'FINANCIAL_RULE: Si se registra
costo_adquisicion en TRANSFERENCIA_INTERNA, el soporte_documental es
obligatorio.'
                USING ERRCODE = 'P0207';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_biologico_origen_financiero
BEFORE INSERT ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_activo_biologico_origen_financiero();
```

TRG-M2-04 — Inmutabilidad de campos estructurales del activo

Descripción	Impide la modificación de `tipo`, `id_especie`, `origen_financiero` y `costo_adquisicion` después del registro inicial. Son campos estructurales inmutables por definición del requerimiento.
Aplicado a	RF-33, RF-35

Restricción origen RF	"El tipo de activo es obligatorio y no puede modificarse después del registro." "El campo origen_financiero_activo no puede modificarse después del registro bajo ninguna circunstancia." "El costo_adquisicion registrado en RF-33 es INMUTABLE." *(RF-33 — Restricciones)*
Justificación	Estos campos son la base de la trazabilidad contable (NIC 41 / RF-82). Si cualquiera cambia después del registro, todos los registros históricos y financieros quedan invalidados. Es la garantía más crítica del módulo.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_activo_biologico_inmutabilidad()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.tipo <> OLD.tipo THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El campo tipo del activo
biológico no puede modificarse después del registro. Valor original:
%.',
            OLD.tipo
            USING ERRCODE = 'P0208';
    END IF;

    IF NEW.id_especie <> OLD.id_especie THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: La especie del activo
biológico no puede modificarse después del registro.'
            USING ERRCODE = 'P0208';
    END IF;

    IF NEW.origen_financiero <> OLD.origen_financiero THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El campo origen_financiero no
puede modificarse después del registro. Es base para la valoración
contable.'
            USING ERRCODE = 'P0208';
    END IF;

    IF (NEW.costo_adquisicion IS DISTINCT FROM OLD.costo_adquisicion)
THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El campo costo_adquisicion es
inmutable. Representa el costo histórico pagado y es base de RF-82.'
            USING ERRCODE = 'P0208';
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_biologico_inmutabilidad
BEFORE UPDATE ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_activo_biologico_inmutabilidad();
```

TRG-M2-05 — Inicialización de métricas del activo poblacional

Descripción	Al insertar un detalle poblacional, fuerza `cantidad_actual = cantidad_inicial`. Garantiza que el campo de referencia histórica esté siempre sincronizado en el origen, independientemente de lo que envíe el backend.
Aplicado a	RF-33, RF-36
Restricción origen RF	"Para activos tipo POBLACIONAL: el campo cantidad_actual debe inicializarse con el valor de cantidad_inicial y no puede ser menor a cero." *(RF-33 — Restricciones)*
Justificación	Sin este trigger, un INSERT con `cantidad_actual` diferente a `cantidad_inicial` crearía un lote cuyo estado inicial es incoherente, rompiendo todos los cálculos posteriores de biomasa y densidad.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_poblacional_inicializar_metricas()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.cantidad_inicial <= 0 THEN
        RAISE EXCEPTION 'INVALID_VALUE: La cantidad_inicial del lote
debe ser mayor a cero. Valor recibido: %.',
```

```
            NEW.cantidad_inicial
            USING ERRCODE = 'P0209';
    END IF;

    -- Forzar cantidad_actual = cantidad_inicial en el registro inicial
    NEW.cantidad_actual := NEW.cantidad_inicial;

    -- cantidad_actual nunca puede ser negativa
    IF NEW.cantidad_actual < 0 THEN
        RAISE EXCEPTION 'INVALID_VALUE: La cantidad_actual no puede ser
negativa.'
            USING ERRCODE = 'P0209';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_poblacional_inicializar_metricas
BEFORE INSERT ON modulo2.detalles_activos_biologicos_poblacionales
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_poblacional_inicializar_metricas();
```

TRG-M2-06 — Inmutabilidad de cantidad_inicial y peso_promedio_inicial del lote

Descripción	Impide cualquier modificación directa de `cantidad_inicial`. Garantiza que el valor histórico de referencia sea permanente. También valida que `cantidad_actual` no sea negativa ni supere `cantidad_inicial + ingresos`.
Aplicado a	RF-33, RF-36
Restricción origen RF	"Para activos POBLACIONAL: los campos cantidad_inicial y peso_promedio_inicial quedan persistidos de forma inmutable en la entidad del activo. Ningún módulo posterior

	puede modificar estos valores." *(RF-33 — Postcondiciones)*
Justificación	`cantidad_inicial` es el punto de referencia histórico consultado por RF-36 en todo momento. Si se puede modificar directamente, la comparación histórica que el sistema muestra al usuario queda falsificada.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_poblacional_cantidad_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    -- cantidad_inicial es inmutable
    IF NEW.cantidad_inicial <> OLD.cantidad_inicial THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El campo cantidad_inicial es
inmutable y no puede modificarse después del registro del lote. Valor
original: %.',
            OLD.cantidad_inicial
            USING ERRCODE = 'P0210';
    END IF;

    -- cantidad_actual no puede ser negativa
    IF NEW.cantidad_actual < 0 THEN
        RAISE EXCEPTION 'INVALID_VALUE: La cantidad_actual del lote no
puede ser negativa. Valor intentado: %.',
            NEW.cantidad_actual
            USING ERRCODE = 'P0210';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_poblacional_cantidad_inmutable
BEFORE UPDATE ON modulo2.detalles_activos_biologicos_poblacionales
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_poblacional_cantidad_inmutable();
```

TRG-M2-07 — Validación de transición de estado del activo biológico

Descripción	Valida que la transición `id_estado_anterior → id_estado_nuevo` sea permitida según la matriz de RF-44. Bloquea en particular cualquier transición desde BAJA y las transiciones inválidas desde CERRADO.
Aplicado a	RF-44
Restricción origen RF	"Desde ACTIVO: puede transitar a INACTIVO, EN_TRATAMIENTO, AISLADO, CERRADO, BAJA. Desde INACTIVO: puede transitar a ACTIVO, EN_TRATAMIENTO, CERRADO, BAJA. Desde EN_TRATAMIENTO: puede transitar a ACTIVO, INACTIVO, AISLADO, CERRADO, BAJA. Desde AISLADO: puede transitar a ACTIVO, INACTIVO, EN_TRATAMIENTO, CERRADO, BAJA. Desde CERRADO: solo puede transitar a BAJA. Desde BAJA: no se permite ninguna transición." *(RF-44 — Descripción)*
Justificación	La matriz de transiciones es el control de ciclo de vida más crítico del módulo. Un UPDATE directo sobre `activos_biologicos.id_estado` saltaría el backend, pero este trigger actúa sobre el historial que es el mecanismo de registro del cambio.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_estado_activo_transicion_valida()
RETURNS TRIGGER AS $$
DECLARE
    v_nombre_anterior VARCHAR(25);
    v_nombre_nuevo    VARCHAR(25);
BEGIN
    SELECT nombre INTO v_nombre_anterior
    FROM modulo2.estados_activos_biologicos
    WHERE id_estado_activo_biologico = NEW.id_estado_anterior;

    SELECT nombre INTO v_nombre_nuevo
    FROM modulo2.estados_activos_biologicos
    WHERE id_estado_activo_biologico = NEW.id_estado_nuevo;
```

```
    -- Desde BAJA no se permite ninguna transición
    IF UPPER(v_nombre_anterior) = 'BAJA' THEN
        RAISE EXCEPTION 'INVALID_TRANSITION: El activo está en estado
BAJA. No se permite ninguna transición desde este estado final
irreversible.'
            USING ERRCODE = 'P0211';
    END IF;

    -- Cambio redundante
    IF NEW.id_estado_anterior = NEW.id_estado_nuevo THEN
        RAISE EXCEPTION 'REDUNDANT_TRANSITION: El estado nuevo es igual
al estado actual. No se realizó ningún cambio.'
            USING ERRCODE = 'P0211';
    END IF;

    -- Validar matriz de transiciones
    IF NOT (
        (UPPER(v_nombre_anterior) = 'ACTIVO'         AND
UPPER(v_nombre_nuevo) IN
('INACTIVO','EN_TRATAMIENTO','AISLADO','CERRADO','BAJA'))
        OR (UPPER(v_nombre_anterior) = 'INACTIVO'    AND
UPPER(v_nombre_nuevo) IN ('ACTIVO','EN_TRATAMIENTO','CERRADO','BAJA'))
        OR (UPPER(v_nombre_anterior) = 'EN_TRATAMIENTO' AND
UPPER(v_nombre_nuevo) IN
('ACTIVO','INACTIVO','AISLADO','CERRADO','BAJA'))
        OR (UPPER(v_nombre_anterior) = 'AISLADO'     AND
UPPER(v_nombre_nuevo) IN
('ACTIVO','INACTIVO','EN_TRATAMIENTO','CERRADO','BAJA'))
        OR (UPPER(v_nombre_anterior) = 'CERRADO'     AND
UPPER(v_nombre_nuevo) IN ('BAJA'))
    ) THEN
        RAISE EXCEPTION 'INVALID_TRANSITION: La transición de "%" a "%"
no está permitida por la matriz de estados del sistema.',
            v_nombre_anterior, v_nombre_nuevo
            USING ERRCODE = 'P0211';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_estado_activo_transicion_valida
BEFORE INSERT ON modulo2.historicos_estados_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_estado_activo_transicion_valida();
```

TRG-M2-08 — Unicidad de estado vigente por activo

Descripción	La tabla `historicos_estados_activos` no tiene un campo `fecha_fin`. La unicidad del estado vigente se infiere como el registro más reciente. Este trigger verifica que al insertar un nuevo registro de estado, el activo efectivamente está en el estado `id_estado_anterior` declarado, garantizando coherencia del historial.
Aplicado a	RF-44
Restricción origen RF	"Cada activo debe tener exactamente un estado vigente en todo momento." *(RF-44 — Restricciones)*
Justificación	Sin esta validación, dos inserts concurrentes podrían registrar estados contradictorios para el mismo activo, rompiendo la integridad del historial.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_estado_activo_unico_vigente()
RETURNS TRIGGER AS $$
DECLARE
    v_ultimo_estado INTEGER;
BEGIN
    -- Obtener el último estado registrado para el activo
    SELECT id_estado_nuevo INTO v_ultimo_estado
    FROM modulo2.historicos_estados_activos
    WHERE id_activo_biologico = NEW.id_activo_biologico
    ORDER BY fecha_cambio DESC
    LIMIT 1;

    -- Si existe historial, el estado anterior declarado debe coincidir
con el último estado registrado
    IF v_ultimo_estado IS NOT NULL AND v_ultimo_estado <>
NEW.id_estado_anterior THEN
        RAISE EXCEPTION 'STATE_INCONSISTENCY: El estado anterior
declarado (ID %) no coincide con el último estado registrado para el
activo (ID %). El historial de estados está desincronizado.',
            NEW.id_estado_anterior, v_ultimo_estado
```

```
            USING ERRCODE = 'P0212';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_estado_activo_unico_vigente
BEFORE INSERT ON modulo2.historicos_estados_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_estado_activo_unico_vigente();
```

TRG-M2-09 — Bloqueo de modificación de activo en estado BAJA

Descripción	Impide cualquier UPDATE sobre un activo biológico que se encuentre en estado BAJA consultando su último estado en `historicos_estados_activos`.
Aplicado a	RF-44, RF-45
Restricción origen RF	"Desde BAJA: no se permite ninguna transición. Estado final irreversible." *(RF-44 — Descripción)* "El activo dado de baja no puede: recibir nuevos eventos, cambiar de estado, participar en procesos productivos." *(RF-45 — Restricciones)*
Justificación	El estado BAJA es irreversible. Sin este trigger, un UPDATE directo sobre el activo podría modificar sus campos incluso después de haber sido dado de baja, invalidando la trazabilidad contable y operativa.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_estado_activo_no_baja_modify()
RETURNS TRIGGER AS $$
DECLARE
    v_ultimo_estado VARCHAR(25);
BEGIN
    SELECT e.nombre INTO v_ultimo_estado
    FROM modulo2.historicos_estados_activos h
```

```
    JOIN modulo2.estados_activos_biologicos e
        ON e.id_estado_activo_biologico = h.id_estado_nuevo
    WHERE h.id_activo_biologico = OLD.id_activo_biologico
    ORDER BY h.fecha_cambio DESC
    LIMIT 1;

    IF UPPER(v_ultimo_estado) = 'BAJA' THEN
        RAISE EXCEPTION 'FINAL_STATE: El activo biológico ID % se
encuentra en estado BAJA definitivo. No se permiten modificaciones sobre
activos dados de baja.',
            OLD.id_activo_biologico
            USING ERRCODE = 'P0213';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_estado_activo_no_baja_modify
BEFORE UPDATE ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_estado_activo_no_baja_modify();
```

TRG-M2-10 — Validación de estado operativo para registro de eventos

Descripción	Verifica que el activo asociado al evento esté en un estado operativo válido: ACTIVO, EN_TRATAMIENTO o AISLADO. Rechaza eventos sobre activos en estado CERRADO o BAJA.
Aplicado a	RF-39, RF-40, RF-41, RF-42, RF-43
Restricción origen RF	"El activo debe encontrarse en un estado operativo que permita el registro de eventos. Los estados operativos válidos son: ACTIVO, EN_TRATAMIENTO, AISLADO. No se permite registrar eventos sobre activos en estado CERRADO o BAJA." *(RF-39 — Precondiciones)*
Justificación	Es el guardián central de la integridad del

	historial. Sin este trigger, eventos podrían registrarse sobre activos ya cerrados o dados de baja, contaminando el historial y los cálculos analíticos.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_evento_activo_estado_valido()
RETURNS TRIGGER AS $$
DECLARE
    v_ultimo_estado VARCHAR(25);
BEGIN
    SELECT e.nombre INTO v_ultimo_estado
    FROM modulo2.historicos_estados_activos h
    JOIN modulo2.estados_activos_biologicos e
        ON e.id_estado_activo_biologico = h.id_estado_nuevo
    WHERE h.id_activo_biologico = NEW.id_activo_biologico
    ORDER BY h.fecha_cambio DESC
    LIMIT 1;

    -- Si no hay historial, verificar el estado directo del activo
    IF v_ultimo_estado IS NULL THEN
        SELECT e.nombre INTO v_ultimo_estado
        FROM modulo2.activos_biologicos a
        JOIN modulo2.estados_activos_biologicos e
            ON e.id_estado_activo_biologico = a.id_estado
        WHERE a.id_activo_biologico = NEW.id_activo_biologico;
    END IF;

    IF UPPER(v_ultimo_estado) IN ('CERRADO', 'BAJA') THEN
        RAISE EXCEPTION 'INVALID_STATE: No es posible registrar eventos
sobre el activo ID %. El activo se encuentra en estado %, el cual no
permite nuevos registros. Estados válidos: ACTIVO, EN_TRATAMIENTO,
AISLADO.',
            NEW.id_activo_biologico, v_ultimo_estado
            USING ERRCODE = 'P0214';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_activo_estado_valido
BEFORE INSERT ON modulo2.eventos_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_activo_estado_valido();
```

TRG-M2-11 — Coherencia temporal de eventos biológicos

Descripción	Valida que la fecha del evento no sea futura y no sea anterior a la `fecha_creacion` del activo biológico.
Aplicado a	RF-39, RF-40, RF-41, RF-42, RF-43
Restricción origen RF	"La fecha del evento no puede ser futura. La fecha del evento no puede ser anterior a la fecha de registro del activo." *(RF-39 — Restricciones)*
Justificación	Es un invariante temporal básico. Un evento con fecha futura o anterior al nacimiento del activo es físicamente imposible y corrompería el orden cronológico del historial.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_evento_fecha_coherente()
RETURNS TRIGGER AS $$
DECLARE
    v_fecha_creacion TIMESTAMPTZ;
BEGIN
    -- Validar que la fecha no sea futura
    IF NEW.fecha > now() THEN
        RAISE EXCEPTION 'INVALID_DATE: La fecha del evento (%) no puede
ser futura. Fecha actual del sistema: %.',
            NEW.fecha, now()
            USING ERRCODE = 'P0215';
    END IF;

    -- Validar que la fecha sea posterior a la creación del activo
    SELECT fecha_creacion INTO v_fecha_creacion
    FROM modulo2.activos_biologicos
    WHERE id_activo_biologico = NEW.id_activo_biologico;

    IF v_fecha_creacion IS NOT NULL AND NEW.fecha < v_fecha_creacion
THEN
        RAISE EXCEPTION 'INVALID_DATE: La fecha del evento (%) no puede
ser anterior a la fecha de registro del activo (%).',
            NEW.fecha, v_fecha_creacion
            USING ERRCODE = 'P0215';
```

```
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_fecha_coherente
BEFORE INSERT ON modulo2.eventos_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_fecha_coherente();
```

TRG-M2-12 — Validación de coherencia del evento de crecimiento por tipo de activo

Descripción	Valida coherencia entre tipo de activo y evento de crecimiento. Para INDIVIDUAL: `tipo_agregacion` debe ser nulo o vacío. Para LOTE: `tipo_agregacion` es obligatorio. También valida coherencia entre `tipo_medicion` y `unidad_medida`.
Aplicado a	RF-40
Restricción origen RF	"Para activos INDIVIDUALES no se permite tipo_agregacion. Para activos LOTE el campo tipo_agregacion es obligatorio. La unidad de medida debe coincidir con el tipo de medición." *(RF-40 — Criterios de aceptación)*
Justificación	Sin esta validación, eventos de crecimiento con datos incoherentes (lote sin agregación, o individual con agregación) generarían recálculos de biomasa incorrectos en TRG-M2-13.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_evento_crecimiento_tipo_activo()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_tipo_activo modulo2.enum_activo_biologico_tipo;
    v_tipo_medicion VARCHAR(55);
    v_unidad VARCHAR(5);
BEGIN
    -- Obtener tipo del activo a través de la FK en eventos_activos
    SELECT a.tipo INTO v_tipo_activo
    FROM modulo2.activos_biologicos a
    JOIN modulo2.eventos_activos ev ON ev.id_activo_biologico =
a.id_activo_biologico
    WHERE ev.id_eventos = NEW.id_evento;

    v_tipo_medicion := UPPER(TRIM(NEW.tipo_medicion));
    v_unidad        := LOWER(TRIM(NEW.unidad_medida));

    -- Validar tipo_agregacion según tipo de activo
    IF v_tipo_activo = 'INDIVIDUAL' THEN
        IF NEW.tipo_agregacion IS NOT NULL AND TRIM(NEW.tipo_agregacion)
<> '' THEN
            RAISE EXCEPTION 'INVALID_FIELD: Para activos INDIVIDUALES el
campo tipo_agregacion debe ser nulo o vacío.'
                USING ERRCODE = 'P0216';
        END IF;
    ELSIF v_tipo_activo = 'POBLACIONAL' THEN
        IF NEW.tipo_agregacion IS NULL OR TRIM(NEW.tipo_agregacion) = ''
THEN
            RAISE EXCEPTION 'MISSING_FIELD: Para activos LOTE
(POBLACIONAL) el campo tipo_agregacion es obligatorio.'
                USING ERRCODE = 'P0216';
        END IF;
    END IF;

    -- Validar valor positivo
    IF NEW.valor_medicion <= 0 THEN
        RAISE EXCEPTION 'INVALID_VALUE: El valor de medición debe ser
positivo y mayor a cero. Valor recibido: %.',
            NEW.valor_medicion
            USING ERRCODE = 'P0217';
    END IF;

    -- Validar coherencia unidad/tipo de medición
    IF v_tipo_medicion = 'PESO' AND v_unidad NOT IN ('kg', 'g', 'lb')
THEN
        RAISE EXCEPTION 'UNIT_MISMATCH: Para medición de PESO solo se
permiten unidades: kg, g, lb. Unidad recibida: %.',
            NEW.unidad_medida
```

```
            USING ERRCODE = 'P0218';
    END IF;

    IF v_tipo_medicion IN ('TALLA', 'LONGITUD', 'ALTURA') AND v_unidad
NOT IN ('cm', 'm') THEN
        RAISE EXCEPTION 'UNIT_MISMATCH: Para medición de
TALLA/LONGITUD/ALTURA solo se permiten unidades: cm, m. Unidad recibida:
%.',
            NEW.unidad_medida
            USING ERRCODE = 'P0218';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_crecimiento_tipo_activo
BEFORE INSERT ON modulo2.eventos_crecimeinto
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_crecimiento_tipo_activo();
```

TRG-M2-13 — Recálculo automático de métricas del lote tras evento de crecimiento

Descripción	Tras registrar un evento de crecimiento sobre un activo POBLACIONAL, actualiza automáticamente `peso_promedio`, `biomasa_total` y `densidad` en `detalles_activos_biologicos_poblacionales`. La superficie para densidad se obtiene de `modulo9.infraestructuras`.
Aplicado a	RF-36, RF-40
Restricción origen RF	"biomasa_total = cantidad_actual × peso_promedio. densidad = cantidad_actual / superficie_infraestructura. La densidad es un valor calculado automáticamente por el sistema." *(RF-36 — Proceso)*

Justificación	Las métricas derivadas deben mantenerse consistentes. Si el backend falla en actualizar las métricas tras registrar un evento de crecimiento, el sistema queda en estado inconsistente con biomasa y densidad desactualizadas.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_evento_crecimiento_recalcular_metricas()
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id     INTEGER;
    v_tipo_activo   modulo2.enum_activo_biologico_tipo;
    v_infra_id      INTEGER;
    v_superficie    NUMERIC(10,2);
    v_cantidad      INTEGER;
    v_nuevo_peso    NUMERIC;
BEGIN
    -- Obtener activo y tipo
    SELECT a.id_activo_biologico, a.tipo, a.id_infraestructura
    INTO v_activo_id, v_tipo_activo, v_infra_id
    FROM modulo2.activos_biologicos a
    JOIN modulo2.eventos_activos ev ON ev.id_activo_biologico =
a.id_activo_biologico
    WHERE ev.id_eventos = NEW.id_evento;

    -- Solo aplica para POBLACIONAL
    IF v_tipo_activo <> 'POBLACIONAL' THEN
        RETURN NEW;
    END IF;

    -- Obtener superficie de la infraestructura (modulo9)
    SELECT superficie INTO v_superficie
    FROM modulo9.infraestructuras
    WHERE id_infraestructura = v_infra_id;

    -- Obtener cantidad_actual del lote
    SELECT cantidad_actual INTO v_cantidad
    FROM modulo2.detalles_activos_biologicos_poblacionales
    WHERE id_activo_biologico = v_activo_id;

    -- El nuevo peso promedio viene del evento (tipo_medicion = PESO con
tipo_agregacion = PROMEDIO)
    IF UPPER(NEW.tipo_medicion) = 'PESO' AND UPPER(NEW.tipo_agregacion)
= 'PROMEDIO' THEN
        v_nuevo_peso := NEW.valor_medicion;
```

```
        UPDATE modulo2.detalles_activos_biologicos_poblacionales
        SET peso_promedio  = v_nuevo_peso,
            biomasa_total  = v_cantidad * v_nuevo_peso,
            densidad       = CASE
                                WHEN v_superficie IS NOT NULL AND
v_superficie > 0
                                THEN v_cantidad::NUMERIC / v_superficie
                                ELSE 0
                             END
        WHERE id_activo_biologico = v_activo_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_crecimiento_recalcular_metricas
AFTER INSERT ON modulo2.eventos_crecimeinto
FOR EACH ROW
EXECUTE FUNCTION
modulo2.trg_fn_evento_crecimiento_recalcular_metricas();
```

TRG-M2-14 — Validación de secuencia lógica del ciclo sanitario

Descripción	Impide registrar TRATAMIENTO o VACUNACION sin un evento de DIAGNOSTICO previo para el mismo activo. Consulta `eventos_activos` + `eventos_sanitarios` para verificar la secuencia.
Aplicado a	RF-41
Restricción origen RF	"No se permite registrar eventos de tratamiento y/o vacunación sin un evento previo de diagnóstico." *(RF-41 — Restricciones)*
Justificación	La secuencia sanitaria es un invariante clínico. Un tratamiento sin diagnóstico previo es incoherente médicamente y viola

	la trazabilidad sanitaria exigida por normativas del ICA.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_evento_sanitario_secuencia()
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id     INTEGER;
    v_tipo_evento   VARCHAR(55);
    v_tiene_diagnostico INTEGER;
BEGIN
    -- Obtener activo del evento padre
    SELECT id_activo_biologico INTO v_activo_id
    FROM modulo2.eventos_activos
    WHERE id_eventos = NEW.id_evento;

    -- Identificar tipo de evento sanitario desde descripcion del evento
padre
    -- Nota: el tipo se infiere del contexto. El DDL actual no tiene
tipo_evento_sanitario
    -- en eventos_sanitarios, por lo que se valida la presencia de
diagnóstico previo
    -- cuando medicamento y dosis están presentes (indica TRATAMIENTO o
VACUNACION)

    IF NEW.medicamento IS NOT NULL AND TRIM(NEW.medicamento) <> ''
       AND NEW.dosis IS NOT NULL AND NEW.dosis > 0 THEN

        SELECT COUNT(*) INTO v_tiene_diagnostico
        FROM modulo2.eventos_sanitarios es_prev
        JOIN modulo2.eventos_activos ea_prev ON ea_prev.id_eventos =
es_prev.id_evento
        WHERE ea_prev.id_activo_biologico = v_activo_id
          AND es_prev.diagnostico IS NOT NULL
          AND TRIM(es_prev.diagnostico) <> ''
          AND es_prev.medicamento IS NULL  -- evento de solo diagnóstico
sin medicamento
          AND ea_prev.fecha < (
              SELECT fecha FROM modulo2.eventos_activos
              WHERE id_eventos = NEW.id_evento
          );

        IF v_tiene_diagnostico = 0 THEN
            RAISE EXCEPTION 'SEQUENCE_VIOLATION: No se puede registrar
TRATAMIENTO o VACUNACION sin un evento de DIAGNOSTICO previo para el
activo ID %.',
```

```
                v_activo_id
                USING ERRCODE = 'P0219';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_sanitario_secuencia
BEFORE INSERT ON modulo2.eventos_sanitarios
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_sanitario_secuencia();
```

TRG-M2-15 — Validación de secuencia del ciclo reproductivo

Descripción	Valida la secuencia del ciclo reproductivo: impide PARTO sin DIAGNOSTICO_GESTACION positivo previo; impide NACIMIENTO sin secuencia completa. Para activos LOTE solo permite tipo NACIMIENTO.
Aplicado a	RF-42
Restricción origen RF	"No se permite registrar nacimientos sin un evento previo de inseminación y diagnóstico de gestación. No se permite registrar parto sin diagnóstico positivo previo. Los activos de tipo lote solo tendrán disponible registrar el evento de tipo Nacimiento." *(RF-42 — Restricciones)*
Justificación	La secuencia reproductiva garantiza integridad genealógica. Sin este trigger, partos sin gestación o nacimientos sin parto previo crearían relaciones genealógicas incoherentes que no pueden corregirse después.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_evento_reproductivo_secuencia()
```

```
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id     INTEGER;
    v_tipo_activo   modulo2.enum_activo_biologico_tipo;
    v_count_previo  INTEGER;
BEGIN
    -- Obtener activo y tipo del evento padre
    SELECT a.id_activo_biologico, a.tipo
    INTO v_activo_id, v_tipo_activo
    FROM modulo2.activos_biologicos a
    JOIN modulo2.eventos_activos ev ON ev.id_activo_biologico =
a.id_activo_biologico
    WHERE ev.id_eventos = NEW.id_evento_reproductivo;

    -- Para LOTE: solo se permite NACIMIENTO
    IF v_tipo_activo = 'POBLACIONAL' AND
       NEW.categoria NOT IN ('NACIMIENTO') THEN
        RAISE EXCEPTION 'TYPE_RESTRICTION: Para activos de tipo LOTE
(POBLACIONAL) solo se permite registrar el evento reproductivo
NACIMIENTO. Categoría recibida: %.',
            NEW.categoria
            USING ERRCODE = 'P0220';
    END IF;

    -- Validar secuencia para activos INDIVIDUALES
    IF v_tipo_activo = 'INDIVIDUAL' THEN

        -- PARTO requiere DIAGNOSTICO_GESTACION previo
        IF NEW.categoria = 'PARTO' THEN
            SELECT COUNT(*) INTO v_count_previo
            FROM modulo2.eventos_reproductivos er
            JOIN modulo2.eventos_activos ea ON ea.id_eventos =
er.id_evento_reproductivo
            WHERE ea.id_activo_biologico = v_activo_id
              AND er.categoria = 'DIAGNOSTICO_GESTACION'
              AND UPPER(er.resultado) LIKE '%POSITIV%';

            IF v_count_previo = 0 THEN
                RAISE EXCEPTION 'SEQUENCE_VIOLATION: No se puede
registrar PARTO sin un evento previo de DIAGNOSTICO_GESTACION positivo
para el activo ID %.',
                    v_activo_id
                    USING ERRCODE = 'P0221';
            END IF;
        END IF;
```

```
        -- NACIMIENTO requiere PARTO previo
        IF NEW.categoria = 'NACIMIENTO' THEN
            SELECT COUNT(*) INTO v_count_previo
            FROM modulo2.eventos_reproductivos er
            JOIN modulo2.eventos_activos ea ON ea.id_eventos =
er.id_evento_reproductivo
            WHERE ea.id_activo_biologico = v_activo_id
              AND er.categoria = 'PARTO';

            IF v_count_previo = 0 THEN
                RAISE EXCEPTION 'SEQUENCE_VIOLATION: No se puede
registrar NACIMIENTO sin un evento previo de PARTO para el activo ID
%.',
                    v_activo_id
                    USING ERRCODE = 'P0221';
            END IF;
        END IF;

        -- Validar número de crías en parto/nacimiento
        IF NEW.categoria IN ('PARTO', 'NACIMIENTO') AND NEW.numero_cria
< 1 THEN
            RAISE EXCEPTION 'INVALID_VALUE: El número de crías en un
evento de % debe ser mayor o igual a 1. Valor recibido: %.',
                NEW.categoria, NEW.numero_cria
                USING ERRCODE = 'P0222';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evento_reproductivo_secuencia
BEFORE INSERT ON modulo2.eventos_reproductivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_reproductivo_secuencia();
```

TRG-M2-16 — Prevención de duplicidad en eventos productivos

Descripción	Impide registrar dos eventos productivos del mismo `id_metrica_produccion` para

	el mismo activo en la misma fecha.
Aplicado a	RF-43
Restricción origen RF	"No se permite registrar dos eventos productivos del mismo tipo_producto para el mismo activo en la misma fecha. El sistema verifica duplicidad antes de persistir." *(RF-43 — Restricciones)*
Justificación	Sin este trigger, datos duplicados de producción (ej. dos registros de leche en el mismo día) inflarían los indicadores zootécnicos y los cálculos financieros de M06.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_evento_productivo_duplicado()
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id INTEGER;
    v_fecha_evento TIMESTAMPTZ;
    v_count INTEGER;
BEGIN
    SELECT ea.id_activo_biologico, ea.fecha
    INTO v_activo_id, v_fecha_evento
    FROM modulo2.eventos_activos ea
    WHERE ea.id_eventos = NEW.id_evento;

    SELECT COUNT(*) INTO v_count
    FROM modulo2.eventos_productivos ep
    JOIN modulo2.eventos_activos ea ON ea.id_eventos = ep.id_evento
    WHERE ea.id_activo_biologico = v_activo_id
      AND ep.id_metrica_produccion = NEW.id_metrica_produccion
      AND DATE(ea.fecha) = DATE(v_fecha_evento);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_EVENT: Ya existe un evento productivo
para la misma métrica (ID %) en el activo ID % en la fecha %. No se
permiten registros duplicados.',
            NEW.id_metrica_produccion, v_activo_id, DATE(v_fecha_evento)
            USING ERRCODE = 'P0223';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_evento_productivo_duplicado
BEFORE INSERT ON modulo2.eventos_productivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_evento_productivo_duplicado();
```

TRG-M2-17 — Validación de cantidad de baja en lote

Descripción	Para activos POBLACIONAL, valida que `cantidad_afectada` no supere `cantidad_actual` del lote y que no sea negativa ni cero.
Aplicado a	RF-45
Restricción origen RF	"En activos tipo lote: se debe permitir baja total o parcial." *(RF-45 — Restricciones)* *"*La cantidad a dar de baja ([CANTIDAD_AFECTADA]) es superior a la existencia actual del lote ([CANTIDAD_ACTUAL])*."* *(RF-45 — Flujo alterno)*
Justificación	Sin esta validación, una baja mayor a la existencia llevaría `cantidad_actual` a negativo, lo que es físicamente imposible y corrompería todos los cálculos de biomasa y densidad.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_baja_cantidad_valida()
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id     INTEGER;
    v_tipo_activo   modulo2.enum_activo_biologico_tipo;
    v_cantidad_actual INTEGER;
BEGIN
    SELECT a.id_activo_biologico, a.tipo
    INTO v_activo_id, v_tipo_activo
    FROM modulo2.activos_biologicos a
    JOIN modulo2.eventos_activos ea ON ea.id_activo_biologico =
a.id_activo_biologico
    WHERE ea.id_eventos = NEW.id_evento;
```

```
    IF v_tipo_activo = 'POBLACIONAL' THEN
        IF NEW.cantidad_afectada IS NULL OR NEW.cantidad_afectada <= 0
THEN
            RAISE EXCEPTION 'INVALID_VALUE: Para bajas en lotes, la
cantidad_afectada debe ser mayor a cero. Valor recibido: %.',
                NEW.cantidad_afectada
                USING ERRCODE = 'P0224';
        END IF;

        SELECT cantidad_actual INTO v_cantidad_actual
        FROM modulo2.detalles_activos_biologicos_poblacionales
        WHERE id_activo_biologico = v_activo_id;

        IF NEW.cantidad_afectada > v_cantidad_actual THEN
            RAISE EXCEPTION 'INVENTORY_INCONSISTENCY: La cantidad a dar
de baja (%) es superior a la existencia actual del lote (%). Activo ID
%.',
                NEW.cantidad_afectada, v_cantidad_actual, v_activo_id
                USING ERRCODE = 'P0225';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_baja_cantidad_valida
BEFORE INSERT ON modulo2.eventos_bajas
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_baja_cantidad_valida();
```

TRG-M2-18 — Actualización automática de cantidad del lote tras baja

Descripción	Tras registrar una baja en un lote POBLACIONAL, descuenta `cantidad_afectada` de `cantidad_actual` y recalcula

	`biomasa_total` y `densidad`.
Aplicado a	RF-36, RF-45
Restricción origen RF	"Eventos de bajas: Reducción de cantidad_actual. Recalculo de métricas. cantidad_actual no puede ser mayor a cantidad_inicial + ingresos registrados. cantidad_actual no puede ser menor a 0." *(RF-36 — Proceso)*
Justificación	Las métricas del lote deben actualizarse atómicamente con la baja. Si el backend falla, el lote quedaría con `cantidad_actual` desactualizada, generando densidades y biomasas incorrectas en todos los módulos que las consumen.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_baja_actualizar_cantidad_lote()
RETURNS TRIGGER AS $$
DECLARE
    v_activo_id     INTEGER;
    v_tipo_activo   modulo2.enum_activo_biologico_tipo;
    v_infra_id      INTEGER;
    v_superficie    NUMERIC(10,2);
    v_nueva_cantidad INTEGER;
    v_peso_promedio  NUMERIC;
BEGIN
    SELECT a.id_activo_biologico, a.tipo, a.id_infraestructura
    INTO v_activo_id, v_tipo_activo, v_infra_id
    FROM modulo2.activos_biologicos a
    JOIN modulo2.eventos_activos ea ON ea.id_activo_biologico =
a.id_activo_biologico
    WHERE ea.id_eventos = NEW.id_evento;

    IF v_tipo_activo <> 'POBLACIONAL' THEN
        RETURN NEW;
    END IF;

    SELECT superficie INTO v_superficie
    FROM modulo9.infraestructuras
    WHERE id_infraestructura = v_infra_id;

    -- Descontar la cantidad y recalcular métricas
    UPDATE modulo2.detalles_activos_biologicos_poblacionales
```

```
    SET cantidad_actual = GREATEST(0, cantidad_actual -
NEW.cantidad_afectada),
        biomasa_total   = GREATEST(0, cantidad_actual -
NEW.cantidad_afectada) * peso_promedio,
        densidad        = CASE
                              WHEN v_superficie IS NOT NULL AND
v_superficie > 0
                              THEN GREATEST(0, cantidad_actual -
NEW.cantidad_afectada)::NUMERIC / v_superficie
                              ELSE 0
                            END
    WHERE id_activo_biologico = v_activo_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_baja_actualizar_cantidad_lote
AFTER INSERT ON modulo2.eventos_bajas
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_baja_actualizar_cantidad_lote();
```

TRG-M2-19 — Unicidad de fase activa por activo biológico

Descripción	Verifica que no exista otra fase con `es_activa = true` para el mismo `id_activo_biologico`. Solo puede existir una fase activa por activo en todo momento.
Aplicado a	RF-37
Restricción origen RF	"Solo puede existir una fase activa por activo. El sistema ejecuta el cambio de fase de forma transaccional." *(RF-37 — Restricciones)*
Justificación	Sin este trigger, dos fases activas simultáneas para el mismo activo harían imposible determinar qué tipo de eventos o productos son válidos, rompiendo RF-43 y los indicadores de RF-51.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_fase_unica_activa()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    IF NEW.es_activa = TRUE THEN
        SELECT COUNT(*) INTO v_count
        FROM modulo2.gestiones_fases
        WHERE id_activo_biologico = NEW.id_activo_biologico
          AND es_activa           = TRUE;

        IF v_count > 0 THEN
            RAISE EXCEPTION 'UNIQUE_ACTIVE_PHASE: El activo biológico ID
% ya tiene una fase productiva activa. Finalice la fase actual antes de
iniciar una nueva.',
                NEW.id_activo_biologico
                USING ERRCODE = 'P0226';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fase_unica_activa
BEFORE INSERT ON modulo2.gestiones_fases
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_fase_unica_activa();
```

TRG-M2-20 — Validación de no solapamiento temporal de fases

Descripción	Valida que la `fecha_inicio` de la nueva fase sea mayor o igual a la `fecha_finalizacion` de la última fase cerrada del activo, evitando solapamientos temporales.
Aplicado a	RF-37
Restricción origen RF	"La fecha_inicio de la nueva fase debe ser

	mayor o igual a la fecha_fin de la fase anterior, garantizando continuidad temporal sin solapamientos. No se permite la existencia de fases con rangos de fechas superpuestos para un mismo activo." *(RF-37 — Restricciones)*
Justificación	Los solapamientos de fases corrompirían los indicadores zootécnicos (RF-51) y la valoración NIC 41 (M06), que dependen de la fase productiva para determinar qué eventos son válidos en cada período.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_fase_solapamiento()
RETURNS TRIGGER AS $$
DECLARE
    v_ultima_fecha_fin TIMESTAMPTZ;
BEGIN
    -- Obtener la fecha de finalización de la última fase del activo
    SELECT fecha_finalizacion INTO v_ultima_fecha_fin
    FROM modulo2.gestiones_fases
    WHERE id_activo_biologico = NEW.id_activo_biologico
      AND es_activa           = FALSE
      AND fecha_finalizacion IS NOT NULL
    ORDER BY fecha_finalizacion DESC
    LIMIT 1;

    -- Si hay fases previas cerradas, la nueva no puede solaparse
    IF v_ultima_fecha_fin IS NOT NULL
       AND NEW.fecha_inicio::TIMESTAMPTZ < v_ultima_fecha_fin THEN
        RAISE EXCEPTION 'PHASE_OVERLAP: La fecha de inicio de la nueva
fase (%) es anterior a la fecha de finalización de la fase anterior (%).
No se permiten solapamientos.',
            NEW.fecha_inicio, v_ultima_fecha_fin
            USING ERRCODE = 'P0227';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fase_solapamiento
BEFORE INSERT ON modulo2.gestiones_fases
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_fase_solapamiento();
```

Nota: gestiones_fases.fecha_inicio tiene tipo time with time zone en el DDL — debería ser timestamptz. El trigger funciona con el tipo actual pero se recomienda corregir el DDL.

TRG-M2-21 — Bloqueo de cambio de fase en activos no operativos

Descripción	Impide registrar un cambio de fase en activos con estado CERRADO o BAJA, consultando el último estado en `historicos_estados_activos`.
Aplicado a	RF-37, RF-38
Restricción origen RF	"No se permite cambio de fase en activos: CERRADOS, DADOS DE BAJA." *(RF-37 — Restricciones)*
Justificación	Un cambio de fase sobre un activo cerrado o dado de baja sería incoherente y podría habilitar el registro de eventos productivos sobre activos que ya no deberían recibirlos.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_fase_activo_estado_valido()
RETURNS TRIGGER AS $$
DECLARE
    v_ultimo_estado VARCHAR(25);
BEGIN
    SELECT e.nombre INTO v_ultimo_estado
    FROM modulo2.historicos_estados_activos h
    JOIN modulo2.estados_activos_biologicos e
        ON e.id_estado_activo_biologico = h.id_estado_nuevo
    WHERE h.id_activo_biologico = NEW.id_activo_biologico
    ORDER BY h.fecha_cambio DESC
    LIMIT 1;

    IF v_ultimo_estado IS NULL THEN
        SELECT e.nombre INTO v_ultimo_estado
        FROM modulo2.activos_biologicos a
        JOIN modulo2.estados_activos_biologicos e
            ON e.id_estado_activo_biologico = a.id_estado
        WHERE a.id_activo_biologico = NEW.id_activo_biologico;
    END IF;
```

```
    IF UPPER(v_ultimo_estado) IN ('CERRADO', 'BAJA') THEN
        RAISE EXCEPTION 'INVALID_STATE: No se puede registrar o cambiar
una fase en el activo ID % porque se encuentra en estado %. Solo se
permiten cambios de fase en activos ACTIVOS.',
            NEW.id_activo_biologico, v_ultimo_estado
            USING ERRCODE = 'P0228';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fase_activo_estado_valido
BEFORE INSERT ON modulo2.gestiones_fases
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_fase_activo_estado_valido();
```

TRG-M2-22 — Unicidad de asociación activa sensor-activo (tipo DIRECTA)

Descripción	Para asociaciones de tipo DIRECTA, verifica que el sensor no tenga ya una asociación activa (sin `fecha_fin`) con otro activo individual. Cardinalidad 1→1 para sensores biométricos individuales.
Aplicado a	RF-49
Restricción origen RF	"Tipo DIRECTA: un sensor biométrico no puede estar activo para más de un activo individual simultáneamente. Debe desactivarse la asociación previa antes de crear una nueva." *(RF-49 — Restricciones)*
Justificación	Sin este trigger, el mismo sensor biométrico podría estar asociado simultáneamente a dos animales distintos, haciendo imposible saber a qué animal pertenece cada lectura de telemetría.

```
CREATE OR REPLACE FUNCTION
modulo2.trg_fn_asociacion_sensor_activo_unica()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_count INTEGER;
BEGIN
    IF NEW.tipo = 'DIRECTA' THEN
        SELECT COUNT(*) INTO v_count
        FROM modulo2.asociaciones_activos_sensores
        WHERE id_sensor   = NEW.id_sensor
          AND fecha_fin   > now()  -- asociación aún vigente
          AND tipo        = 'DIRECTA';

        IF v_count > 0 THEN
            RAISE EXCEPTION 'SENSOR_CONFLICT: El sensor ID % ya tiene
una asociación DIRECTA activa con otro activo individual. Desactive la
asociación existente antes de crear una nueva.',
                NEW.id_sensor
                USING ERRCODE = 'P0229';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_asociacion_sensor_activo_unica
BEFORE INSERT ON modulo2.asociaciones_activos_sensores
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_asociacion_sensor_activo_unica();
```

TRG-M2-23 — Inmutabilidad del historial de estados

Descripción	Bloquea cualquier UPDATE o DELETE sobre el historial de estados. El historial es append-only e inmutable.
Aplicado a	RF-44, RF-52
Restricción origen RF	"El historial de estados es inmutable (sin UPDATE ni DELETE sobre registros de estado ya registrados)." *(RF-44 — Restricciones)*
Justificación	El historial de estados es evidencia auditada

	del ciclo de vida del activo. Si puede modificarse, toda la trazabilidad contable y normativa queda comprometida. Debe ser la restricción más robusta del módulo.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_historial_estados_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'IMMUTABLE_RECORD: El historial de estados del
activo biológico es inmutable. Operación % bloqueada. El historial es
append-only.',
        TG_OP
        USING ERRCODE = 'P0230';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_historial_estados_inmutable_update
BEFORE UPDATE ON modulo2.historicos_estados_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_historial_estados_inmutable();

CREATE TRIGGER trg_historial_estados_inmutable_delete
BEFORE DELETE ON modulo2.historicos_estados_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_historial_estados_inmutable();
```

TRG-M2-24 — Inmutabilidad de todos los eventos biológicos registrados

Descripción	Bloquea cualquier UPDATE o DELETE sobre eventos ya registrados. Los eventos biológicos son inmutables una vez persistidos.
Aplicado a	RF-39 al RF-45, RF-52
Restricción origen RF	"No se permite la eliminación de eventos registrados; solo se permite su consulta." *(RF-39 — Restricciones)*
Justificación	Los eventos biológicos son la evidencia de

	la transformación biológica del activo (NIC 41). Si pueden modificarse o eliminarse, la cadena de custodia para auditorías del ICA y la UPRA queda invalidada.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_eventos_activos_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'IMMUTABLE_RECORD: Los eventos biológicos son
inmutables una vez registrados. Operación % sobre tabla % bloqueada.',
        TG_OP, TG_TABLE_NAME
        USING ERRCODE = 'P0231';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a eventos_activos
CREATE TRIGGER trg_eventos_activos_inmutable_update
BEFORE UPDATE ON modulo2.eventos_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_activos_inmutable_delete
BEFORE DELETE ON modulo2.eventos_activos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

-- Aplicar a eventos_crecimeinto
CREATE TRIGGER trg_eventos_crecimiento_inmutable_update
BEFORE UPDATE ON modulo2.eventos_crecimeinto
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_crecimiento_inmutable_delete
BEFORE DELETE ON modulo2.eventos_crecimeinto
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

-- Aplicar a eventos_sanitarios
CREATE TRIGGER trg_eventos_sanitarios_inmutable_update
BEFORE UPDATE ON modulo2.eventos_sanitarios
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_sanitarios_inmutable_delete
```

```
BEFORE DELETE ON modulo2.eventos_sanitarios
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

-- Aplicar a eventos_reproductivos
CREATE TRIGGER trg_eventos_reproductivos_inmutable_update
BEFORE UPDATE ON modulo2.eventos_reproductivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_reproductivos_inmutable_delete
BEFORE DELETE ON modulo2.eventos_reproductivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

-- Aplicar a eventos_productivos
CREATE TRIGGER trg_eventos_productivos_inmutable_update
BEFORE UPDATE ON modulo2.eventos_productivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_productivos_inmutable_delete
BEFORE DELETE ON modulo2.eventos_productivos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

-- Aplicar a eventos_bajas
CREATE TRIGGER trg_eventos_bajas_inmutable_update
BEFORE UPDATE ON modulo2.eventos_bajas
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();

CREATE TRIGGER trg_eventos_bajas_inmutable_delete
BEFORE DELETE ON modulo2.eventos_bajas
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_eventos_activos_inmutable();
```

TRG-M2-25 — Fecha de inicio de ciclo productivo válida

Descripción	Valida que `fecha_inicio_ciclo` no sea futura y no sea anterior a 1970-01-01

	(fecha mínima válida del requerimiento). También detecta el gap de tipo de dato: el campo está como `integer` en el DDL pero debería ser `date`.
Aplicado a	RF-33
Restricción origen RF	"La fecha_inicio_ciclo es obligatoria, no puede ser posterior a la fecha actual del sistema ni inferior a una fecha mínima válida (1970-01-01)." *(RF-33 — Restricciones)*
Justificación	La fecha de inicio del ciclo es la base temporal de todos los eventos históricos. Una fecha futura o anterior al mínimo produciría inconsistencias en el historial desde el origen.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_activo_fecha_inicio_valida()
RETURNS TRIGGER AS $$
BEGIN
    -- El campo está como integer en el DDL; validar que no sea negativo
(anterior a 1970)
    -- y no sea futuro (epoch actual en días)
    IF NEW.fecha_inicio_ciclo IS NULL THEN
        RAISE EXCEPTION 'MISSING_FIELD: El campo fecha_inicio_ciclo es
obligatorio.'
            USING ERRCODE = 'P0232';
    END IF;

    -- Valor negativo = anterior a 1970-01-01
    IF NEW.fecha_inicio_ciclo < 0 THEN
        RAISE EXCEPTION 'INVALID_DATE: La fecha_inicio_ciclo no puede
ser anterior a 1970-01-01.'
            USING ERRCODE = 'P0232';
    END IF;

    -- Valor futuro = epoch en días mayor al actual
    IF NEW.fecha_inicio_ciclo > EXTRACT(EPOCH FROM now())::INTEGER /
86400 THEN
        RAISE EXCEPTION 'INVALID_DATE: La fecha_inicio_ciclo no puede
ser posterior a la fecha actual del sistema.'
            USING ERRCODE = 'P0232';
    END IF;

    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_fecha_inicio_valida
BEFORE INSERT ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_activo_fecha_inicio_valida();
```

Nota: El campo fecha_inicio_ciclo está declarado como integer en el DDL. El trigger lo trata como días desde época Unix para la validación hasta que el DDL sea corregido a date.

TRG-M2-26 — Protección de eliminación física de activos biológicos

Descripción	Bloquea cualquier DELETE físico sobre activos biológicos. Solo se permite la gestión del estado mediante `historicos_estados_activos`. La eliminación lógica es mediante estado BAJA.
Aplicado a	RF-35, RF-36, RF-44, RF-52
Restricción origen RF	"No se permite eliminar activos, solo marcar su estado (Activo, Inactivo, Cerrado, Baja)." *(RF-35 — Restricciones)*
Justificación	Los activos biológicos son la entidad central del módulo. Eliminarlos físicamente rompería todas las FK de eventos, historial de estados, fases e indicadores, además de destruir la trazabilidad NIC 41 requerida.

```
CREATE OR REPLACE FUNCTION modulo2.trg_fn_activo_biologico_no_delete()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'NO_PHYSICAL_DELETE: Los activos biológicos no
pueden eliminarse físicamente. Use la gestión de estados (BAJA) para
retirar un activo del sistema. Activo ID: %.',
        OLD.id_activo_biologico
        USING ERRCODE = 'P0233';
    RETURN NULL;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activo_biologico_no_delete
BEFORE DELETE ON modulo2.activos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo2.trg_fn_activo_biologico_no_delete();
```